```
#include <stdio.h>
#include <stdlib.h>

// Structure for tree node
struct Node {
    int data;
    struct Node *left, *right;
};

// Create new node
struct Node* createNode(int data) {
    struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));
    newNode->data = data;
    newNode->left = newNode->right = NULL;
    return newNode;
}

////////////////////////////////////////////////
//  🔹 RECURSIVE TRAVERSALS
////////////////////////////////////////////////

void inorderRecursive(struct Node* root) {
    if (root != NULL) {
        inorderRecursive(root->left);
        printf("%d ", root->data);
        inorderRecursive(root->right);
    }
}

void preorderRecursive(struct Node* root) {
    if (root != NULL) {
        printf("%d ", root->data);
        preorderRecursive(root->left);
        preorderRecursive(root->right);
    }
}

void postorderRecursive(struct Node* root) {
    if (root != NULL) {
        postorderRecursive(root->left);
```

```
        postorderRecursive(root->right);
        printf("%d ", root->data);
    }
}

////////////////////////////////////////////////
//  🔹 NON-RECURSIVE TRAVERSALS (Using Stack)
////////////////////////////////////////////////

#define MAX 100

struct Node* stack[MAX];
int top = -1;

void push(struct Node* node) {
    stack[++top] = node;
}

struct Node* pop() {
    return stack[top--];
}

int isEmpty() {
    return top == -1;
}

// Non-Recursive Inorder
void inorderNonRecursive(struct Node* root) {
    struct Node* current = root;

    while (current != NULL || !isEmpty()) {
        while (current != NULL) {
            push(current);
            current = current->left;
        }

        current = pop();
        printf("%d ", current->data);
        current = current->right;
    }
```

```
}

// Non-Recursive Preorder
void preorderNonRecursive(struct Node* root) {
    if (root == NULL) return;

    push(root);

    while (!isEmpty()) {
        struct Node* temp = pop();
        printf("%d ", temp->data);

        if (temp->right)
            push(temp->right);
        if (temp->left)
            push(temp->left);
    }
}

// Non-Recursive Postorder (using two stacks)
void postorderNonRecursive(struct Node* root) {
    if (root == NULL) return;

    struct Node* stack1[MAX];
    struct Node* stack2[MAX];
    int top1 = -1, top2 = -1;

    stack1[++top1] = root;

    while (top1 != -1) {
        struct Node* temp = stack1[top1--];
        stack2[++top2] = temp;

        if (temp->left)
            stack1[++top1] = temp->left;
        if (temp->right)
            stack1[++top1] = temp->right;
    }

    while (top2 != -1) {
```

```
        printf("%d ", stack2[top2--]->data);
    }
}

////////////////////////////////////////////////
//  🔹 MAIN FUNCTION
////////////////////////////////////////////////

int main() {
    // Creating sample tree
    /*
           1
          / \
         2   3
        / \
       4   5
    */

    struct Node* root = createNode(1);
    root->left = createNode(2);
    root->right = createNode(3);
    root->left->left = createNode(4);
    root->left->right = createNode(5);

    printf("Recursive Traversals:\n");

    printf("Inorder: ");
    inorderRecursive(root);

    printf("\nPreorder: ");
    preorderRecursive(root);

    printf("\nPostorder: ");
    postorderRecursive(root);

    printf("\n\nNon-Recursive Traversals:\n");

    printf("Inorder: ");
    inorderNonRecursive(root);
```

```
    printf("\nPreorder: ");
    preorderNonRecursive(root);

    printf("\nPostorder: ");
    postorderNonRecursive(root);

    return 0;
}
```